Game Theory Eleven

Jacob Wyngaard

Contents

1 BNN Error Orders

First, I continued the numerical study of Brown–von Neumann–Nash (BNN) dynamics for two-player, two-strategy games. In particular, I further investigated the example game in which neither player has a strictly dominant strategy. In this game, the payoff matrices are

$$A = \begin{pmatrix} 2 & 3 \\ 4 & 1 \end{pmatrix}, \qquad B = \begin{pmatrix} 7 & 5 \\ 6 & 8 \end{pmatrix}.$$

Here A is the payoff matrix for Player 1 and B is the payoff matrix for Player 2.

The mixed strategies are written as

$$p^1 = (x^1, y^1), \qquad p^2 = (x^2, y^2),$$

where p^1 is Player 1's mixed strategy and p^2 is Player 2's mixed strategy. Since each player has two strategies, it is enough to track x^1 and x^2, with

$$y^1 = 1 - x^1, \qquad y^2 = 1 - x^2.$$

1.1 Comparison of RK4 to an Adaptive Runge–Kutta Solution

I compared the fourth-order Runge–Kutta approximation against a high-accuracy "actual solution" computed using an adaptive Runge–Kutta (RK-45) method. The numerical experiment starts at the mixed strategy profile

$$x_1(0) = \frac{1}{3}, \qquad y_1(0) = \frac{1}{3}.$$

The solution is then evolved over a fixed time interval

$$t \in [0, T].$$

In the earlier numerical experiments, the RK4 error appeared to behave like a lower-order method with noise in the no-strict-dominance case. This week, I observed that the apparent order of convergence depends strongly on whether the solution crosses a region where the Jacobian of the BNN vector field changes.

1.2 Why the No-Strict-Dominance Case is Delicate

For BNN dynamics, the vector field involves rectified terms through the definition of the ϕ functions. In this two-strategy case, this can create transitions at boundaries such as

$$x^1 = \frac{1}{2}, \qquad x^2 = \frac{1}{2}.$$

At these transition planes, the Jacobian of the vector field changes. I conjecture that, for a general two player game, each player's Nash equilibrium strategies produce transition planes. This will be investigated further next week, when both players will have three strategies.

The transition planes affect RK4 because classical fourth-order convergence assumes sufficient smoothness of the vector field. When the trajectory crosses a point where the Jacobian changes abruptly, the smoothness assumption is violated and the observed error order degrades.

1.3 Restricting the Time Interval

When the system is considered for a period of time in which it stays in the same quadrant, the error order reverts to the expected fourth order. The empirical conclusion is:

> A consistent Jacobian ensures the proper error order, while an inconsistent Jacobian interferes with the observed error order.

Thus, when the numerical trajectory remains in a region where the same branch of the BNN vector field is active, RK4 displays the expected fourth-order behavior. When the trajectory passes through a non-smooth Jacobian switching region, the observed convergence can look noisy and/or closer to second order.

1.4 Rectified Quadratic Unit

I did a quick investigation with the rectified quadratic unit

$$q(z) = \big(\max\{z, 0\}\big)^2.$$

This function is continuously differentiable, but its second derivative changes at $z = 0$. In BNN dynamics, this rectified structure can produce a vector field that is smooth on each active region but not globally smooth in the sense required for clean RK4 error estimates across all time intervals. This was reflected in the noisy and/or closer to second order errors that the rectified quadratic unit method produced.

2 Compressible Navier–Stokes

The second part of this week's work focused on the relationship between compressible Navier–Stokes equations and mean field games.

For a compressible fluid, the main variables are:

$$v(t, x) = \text{fluid velocity}, \qquad \rho(t, x) = \text{fluid density}.$$

The density is not assumed to be constant, so we must track both the flow of the fluid and the evolution of the density.

2.1 Flow Equation

The compressible Navier–Stokes momentum equation has the general form

$$(\partial_t + v \cdot \nabla)v = \frac{-\nabla p + \mu \Delta v + f}{\rho}.$$

2.2 Density Equation

The density evolves according to conservation of mass:

$$\partial_t \rho + \nabla \cdot (\rho v) = 0.$$

The term ρv is the mass flux, so $\nabla \cdot (\rho v)$ measures the net flow of mass out of a local region.

2.3 Compressible Navier–Stokes System

Putting the velocity and density equations together gives

$$\boxed{\begin{aligned} (\partial_t + v \cdot \nabla)v &= \frac{-\nabla p + \mu \Delta v + f}{\rho}, \\ \partial_t \rho + \nabla \cdot (\rho v) &= 0. \end{aligned}}$$

This system is usually paired with initial data

$$v(0, x) = v_0(x), \qquad \rho(0, x) = \rho_0(x).$$

3 Numerically Modeling Compressible Navier–Stokes

I numerically modeled the compressible Navier–Stokes equations using:

- second-order finite difference methods for spatial derivatives,
- forward Euler time stepping,
- JAX Python library to compile the update steps into optimized machine code.

The relevant Python code is attached in Appendix A

3.1 Stability Conditions

The explicit methods used are unstable when the time step is too large relative to the spatial grid size, velocity scale, or viscosity.

Stability can be achieved by satisfying the following three constraints:

$$\text{convective condition}, \qquad \text{diffusive condition}, \qquad \text{compressibility condition}.$$

The convective condition is

$$\Delta t \lesssim \frac{\Delta x}{\max |v|}.$$

The diffusive condition is

$$\Delta t \lesssim C\frac{\Delta x^2}{\mu},$$

where ν is the viscosity

The compressibility condition is

$$\Delta t \lesssim \frac{\Delta x}{\max\sqrt{\frac{\partial p}{\partial \rho}}}.$$

The convective and compressibility are normally combined into

$$\Delta t \lesssim \frac{\Delta x}{\max\sqrt{\frac{\partial p}{\partial \rho}} + \max|v|}.$$

4 Luo and Song's Compressible Navier–Stokes Variation

Luo and Song, in their paper connecting Navier-Stokes to Mean Field Game Theory, studied a modified version of compressible Navier-Stokes. One of their key changes was to add a diffusive noise term to the density evolution.

The modified system has the form

$$\boxed{\begin{aligned}&(\partial_t + v\cdot\nabla)v = -\nabla p[\rho] + \mu[\rho]\Delta v + f[\rho],\\ &\partial_t\rho - \nabla\cdot(\rho v) - \mu[\rho]\Delta\rho = 0\end{aligned}}$$

Here $p[\rho]$, $\mu[\rho]$, and $f[\rho]$ may depend functionally on the density.

Luo and Song were mainly interested in the following case,

$$\mu[\rho] = \frac{1}{2}, \qquad f[\rho] = 0, \qquad v_0(x) = \nabla h(x),$$

where the system becomes

$$\boxed{\begin{aligned}&(\partial_t + v\cdot\nabla)v = -\nabla p[\rho] + \tfrac{1}{2}\Delta v,\\ &\partial_t\rho - \nabla\cdot(\rho v) - \tfrac{1}{2}\Delta\rho = 0,\\ &v(0,x) = \nabla h[\rho](x),\\ &\rho(T,x) = \rho_T(x).\end{aligned}}$$

4.1 Initial and Terminal Conditions

A key feature of the Luo–Song system is that it is a coupled forward-backward system:

$$v(0,x) = v_0(x), \qquad \rho(T,x) = \rho_T(x).$$

The flow equation has an initial condition, while the density equation has a terminal condition. This is one reason the system resembles the forward-backward PDE systems that appear in mean field games (the system Lasry and Lions described).

5 Fixed-Point Iteration for the Luo–Song System

To solve this coupled forward-backward system, I used a Banach–Picard, or fixed-point, iteration strategy.

The strategy is:

1. Guess an initial density path
$$\rho^{(0)}(t,x).$$
2. Given $\rho^{(k)}$, solve the velocity equation forward in time:
$$(\partial_t + v^{(k+1)} \cdot \nabla)v^{(k+1)} = -\nabla p[\rho^{(k)}] + \frac{1}{2}\Delta v^{(k+1)}.$$
3. Use the resulting velocity path $v^{(k+1)}$ to solve the density equation:
$$\partial_t \rho^{(k+1)} - \nabla \cdot (\rho^{(k+1)} v^{(k+1)}) - \frac{1}{2}\Delta \rho^{(k+1)} = 0.$$
4. Repeat until the density path converges:
$$\|\rho^{(k+1)} - \rho^{(k)}\| < \varepsilon.$$

5.1 Why Explicit Time Reversal is Unstable

The density equation contains a diffusion term. Written forward in time, a heat equation has the form
$$\partial_t \rho = \mu \Delta \rho.$$
which is stable. But if we reverse time, we obtain the backward heat equation
$$\partial_t \rho = -\mu \Delta \rho,$$
which is ill-posed as high-frequency modes grow exponentially instead of decaying.

This is why papers like Camilli's investigation of policy iteration for mean field games have the flow equation with the terminal condition and the density equation with the initial condition.

6 Compressible Navier–Stokes Models versus Mean Field Games

A representative mean field game system is

$$\boxed{\begin{aligned} &\partial_t u - \mu \Delta u + H(x, \nabla u) = W(m), \\ &\partial_t m + \mu \Delta m + \nabla \cdot \left(m \frac{\partial H}{\partial p}(x, \nabla u) \right) = 0 \end{aligned}}$$

The value function is $u(t,x)$ and the population density is $m(t,x)$.

The Luo–Song system can be compared to the MFG system by setting

$$v = \nabla u, \qquad m = \rho, \qquad \mu = \frac{1}{2}, \qquad H(x, \nabla u) = \frac{1}{2}|\nabla u|^2.$$

Then

$$\frac{\partial H}{\partial p}(x, p) = p,$$

so at $p = \nabla u$,

$$\frac{\partial H}{\partial p}(x, \nabla u) = \nabla u.$$

The density equation becomes

$$\partial_t m + \nu \Delta m + \nabla \cdot (m \nabla u) = 0,$$

which matches the structure of the Luo–Song density equation.

7 Hamiltonian Redux

The Hamiltonian encodes the best response of an agent given the cost structure of the problem. In an HJB equation, the Hamiltonian is obtained by optimizing over controls.

Consider the objective structure

$$J(t, x, a) = \mathbb{E}\left[\int_t^T I(s, X_s, a_s)\,\mathrm{d}s + C(X_T)\right],$$

where the controlled state evolves as

$$\mathrm{d}X_s = g(s, X_s, a_s)\,\mathrm{d}s + \sigma(X_s)\,\mathrm{d}W_s.$$

The value function is

$$V(t, x) = \sup_{a(\cdot)} J(t, x, a).$$

The stochastic HJB equation is

$$-\partial_t V = \sup_a \left[I(t, x, a) + \nabla V \cdot g(t, x, a)\right] + \frac{1}{2}\sigma^2 \Delta V.$$

The Hamiltonian is therefore

$$H(x, \nabla V) = \sup_a \left[I(t, x, a) + \nabla V \cdot g(t, x, a)\right].$$

7.1 Quadratic Control Cost

A common modeling assumption is:

$$I(t, x, a) = -\frac{1}{2}|a|^2, \qquad g(t, x, a) = a.$$

This means the agent has complete control over its rate of movement, but moving quickly is costly.

Then the Hamiltonian is

$$H(\nabla V) = \sup_a \left[\nabla V \cdot a - \frac{1}{2}|a|^2 \right].$$

The first-order condition is

$$\nabla V - a = 0,$$

so the optimal control is

$$a^* = \nabla V.$$

Substituting back,

$$H(\nabla V) = \nabla V \cdot \nabla V - \frac{1}{2}|\nabla V|^2 = \frac{1}{2}|\nabla V|^2.$$

Thus a quadratic control cost produces the Hamiltonian

$$\boxed{H(p) = \frac{1}{2}|p|^2.}$$

8 Numerically Modeling Mean Field Games

A mean field game system couples a value PDE and a density PDE. In the time convention used in the slides, the value equation is solved backward in time and the density equation is solved forward in time:

$$\boxed{\begin{aligned} &\partial_t u - \mu \Delta u + H(x, \nabla u) = W(m), \\ &\partial_t m + \mu \Delta m + \nabla \cdot \left(m \frac{\partial H}{\partial p}(x, \nabla u) \right) = 0 \end{aligned}}$$

Here:

- $u(t, x)$ is the value function of a representative player;
- $m(t, x)$ is the distribution of players across states;
- $H(x, \nabla u)$ encodes optimal individual behavior;
- $W(m)$ is the interaction cost or payoff induced by the population.

For the quadratic Hamiltonian

$$H(p) = \frac{1}{2}|p|^2,$$

we have

$$\frac{\partial H}{\partial p}(p) = p.$$

Therefore the density equation becomes

$$\partial_t m + \nu \Delta m + \nabla \cdot (m \nabla u) = 0.$$

9 Financial Example: Collectible Card Market

Suppose a group of traders are collecting a specific type of Pokémon card.

Let

$$x = \text{number of cards owned by a trader.}$$

Then

$$u(t, x)$$

represents the value of owning x copies of the card at time t, while

$$m(t, x)$$

represents the distribution of inventories across traders.

The stochastic time evolution can be modeled as

$$\mathrm{d}X_t = a_t \,\mathrm{d}t + \sigma \,\mathrm{d}W_t.$$

Here:

- $a_t > 0$ means the trader buys cards;
- $a_t < 0$ means the trader sells cards;
- $\sigma \,\mathrm{d}W_t$ represents random shocks, such as finding cards in packs or losing cards.

9.1 Quadratic Trading Cost

Assume that buying or selling cards has a quadratic cost:

$$\frac{1}{2}|a|^2.$$

This captures the idea that aggressive trading moves the price against the trader. The objective structure is

$$I(t, x, a, m) = -\frac{1}{2}|a|^2 + W(m).$$

The associated Hamiltonian is

$$H(p) = \sup_a \left[p \cdot a - \frac{1}{2}|a|^2 \right] = \frac{1}{2}|p|^2.$$

Therefore the HJB equation becomes

$$-\partial_t V = \frac{1}{2}|\nabla V|^2 + W(m) + \mu \Delta V.$$

Equivalently, after changing to V to u, we obtain

$$\partial_t u - \mu \Delta u + \frac{1}{2}|\nabla u|^2 = W(m).$$

The coupled population equation is

$$\partial_t m + \mu \Delta m + \nabla \cdot (m \nabla u) = 0.$$

Together, these equations describe how individual traders optimally buy or sell based on the current market distribution, while the market distribution evolves in response to those individual decisions.

A more interesting example I want to investigate is bubbles in financial markets. The interaction term could be influenced by the "Greater Fools Theorem" and the stopping time could be a random variable. I think that this could be a fascinating example!

A 1D Compressible Navier–Stokes JAX Code

```python
import jax
import jax.numpy as jnp
import numpy as np
import matplotlib.pyplot as plt

# Parameters
N = 400
L = 1.0
dx = L/N

T = .21
dt = 2e-6
steps = int(T/dt)

viscosity = 1/2
assert(dt <= dx**2/2/viscosity) # Diffusive Condition

x = jnp.linspace(0, L, N, endpoint=False)

# Initial Condition
rho = (
    1.0
    + 0.25*jnp.exp(-200*(x - 0.25)**2)
    + 0.25*jnp.exp(-200*(x - 0.75)**2)
)

u = jnp.zeros_like(x)

# Finite Differences [Second Order]
def ddx(f):
    return (jnp.roll(f, -1) - jnp.roll(f, 1))/(2*dx)

def d2dx2(f):
    return (jnp.roll(f, -1) - 2*f + jnp.roll(f, 1))/dx**2

gamma = .5

def pressure(rho):
    return rho**gamma

# One Time Step
@jax.jit
def step(rho, u):
    p = pressure(rho)

    # Flow
    u_t = (
        -u*ddx(u)
        + viscosity*d2dx2(u)
        - (1/rho)*ddx(p)
    )
```

```
52
53      # Density
54      rho_t = -ddx(rho*u)
55
56      # Update
57      rho_new = rho + dt*rho_t
58      u_new = u + dt*u_t
59
60      # Minimum Density
61      rho_new = jnp.maximum(rho_new, 1e-6)
62
63      return rho_new, u_new
64
65  # Run Simulation
66  plt.figure()
67
68  for n in range(steps):
69      rho, u = step(rho, u)
70
71      c = jnp.sqrt(gamma*rho**(gamma - 1))
72      max_wave_speed = jnp.max(jnp.abs(u) + c)
73      assert(dt <= dx/max_wave_speed)
74
75      if n % 50000 == 0:
76          plt.clf()
77          plt.plot(np.array(x), np.array(rho), label=r"$\rho$")
78          plt.plot(np.array(x), np.array(u), label=r"$v$")
79          plt.ylim(-0.5, 1.5)
80          plt.title(f"1D compressible Navier-Stokes, t = {n*dt:.4f}")
81          plt.xlabel("x")
82          plt.legend()
83          plt.pause(0.01)
84
85  plt.show()
```